

Unit D1

Decision Mathematics 1

GCE AS and GCE Mathematics, GCE AS and GCE Further Mathematics and GCE AS and GCE Further Mathematics (Additional) AS optional unit

D1.1 Unit description

Algorithms; algorithms on graphs; the route inspection problem; critical path analysis; linear programming; matchings.

D1.2 Assessment information

Preamble

Students should be familiar with the terms defined in the glossary attached to this specification. Students should show clearly how an algorithm has been applied. Matrix representation will be required but matrix manipulation is not required. Students will be required to model and interpret situations, including cross-checking between models and reality.

Examination

The examination will consist of one 1½ hour paper. The paper will contain about seven questions with varying mark allocations per question which will be stated on the paper. All questions may be attempted.

Calculators

Students are expected to have available a calculator with at least the following keys: $+$, $-$, $\times$, $\div$, π , x^2 , $\sqrt{x}$, $\frac{1}{x}$, x^y and memory. Calculators with a facility for symbolic algebra, differentiation and/or integration are not permitted.

Formulae

Students are expected to know any other formulae which might be required by the specification and which are not included in the booklet, *Mathematical Formulae including Statistical Formulae and Tables*, which will be provided for use with the paper. Questions will be set in SI units and other units in common usage.

1 Algorithms

What students need to learn:

The general ideas of algorithms and the implementation of an algorithm given by a flow chart or text.

The order of an algorithm is not expected.

Whenever finding the middle item of any list, the method defined in the glossary must be used.

Students should be familiar with bin packing, bubble sort, quick sort, binary search.

When using the quick sort algorithm, the pivot should be chosen as the middle item of the list.

2 Algorithms on graphs

What students need to learn:

The minimum spanning tree (minimum connector) problem. Prim's and Kruskal's (greedy) algorithm.

Matrix representation for Prim's algorithm is expected. Drawing a network from a given matrix and writing down the matrix associated with a network will be involved.

Dijkstra's algorithm for finding the shortest path.

3 The route inspection problem

What students need to learn:

Algorithm for finding the shortest route around a network, travelling along every edge at least once and ending at the start vertex. The network will have up to four odd nodes.

Also known as the 'Chinese postman' problem. Students will be expected to use inspection to consider all possible pairings of odd nodes.

(The application of Floyd's

4 Critical path analysis

What students need to learn:

Modelling of a project by an activity network, from a precedence table.

Completion of the precedence table for a given activity network.

Algorithm for finding the critical path. Earliest and latest event times. Earliest and latest start and finish times for activities.

Total float. Gantt (cascade) charts. Scheduling.

Activity on arc will be used. The use of dummies is included. In a precedence network, precedence tables will only show immediate predecessors.

5 Linear programming

What students need to learn:

Formulation of problems as linear programs.

Graphical solution of two variable problems using ruler and vertex methods

Consideration of problems where solutions must have integer values.

6 Matchings

What students need to learn:

Use of bipartite graphs for modelling matchings. Complete matchings and maximal matchings.

Students will be required to use the maximum matching algorithm to improve a matching by finding alternating paths. No consideration of assignment is required.

Glossary for D1

1 Algorithms

In a list containing N items the 'middle' item has position $\lceil \frac{1}{2}(N+1) \rceil$ if N is odd $\lceil \frac{1}{2}(N+2) \rceil$ if N is even, so that if $N=9$, the middle item is the 5th and if $N=6$ it is the 4th.

2 Algorithms on graphs

A **graph** G consists of points (**vertices** or **nodes**) which are connected by lines (**edges** or **arcs**).

A **subgraph** of G is a graph, each of whose vertices belongs to G and each of whose edges belongs to G .

If a graph has a number associated with each edge (usually called its **weight**) then the graph is called a **weighted graph** or **network**.

The **degree** or **valency** of a vertex is the number of edges incident to it. A vertex is **odd (even)** if it has **odd (even)** degree.

A **path** is a finite sequence of edges, such that the end vertex of one edge in the sequence is the start vertex of the next, and in which no vertex appears more than once.

A **cycle (circuit)** is a closed path, ie the end vertex of the last edge is the start vertex of the first edge.

Two vertices are **connected** if there is a path between them. A graph is **connected** if all its vertices are connected.

If the edges of a graph have a direction associated with them they are known as **directed edges** and the graph is known as a **digraph**.

A **tree** is a connected graph with no cycles.

A **spanning tree** of a graph G is a subgraph which includes all the vertices of G and is also a tree.

A **minimum spanning tree** (MST) is a spanning tree such that the total length of its arcs is as small as possible. (MST is sometimes called a **minimum connector**.)

A graph in which each of the n vertices is connected to every other vertex is called a **complete graph**.

4 Critical path analysis

The **total float** $F(i, j)$ of activity (i, j) is defined to be $F(i, j) = l_j - e_i - \text{duration}(i, j)$, where e_i is the earliest time for event i and l_j is the latest time for event j .

6 Matchings

A **bipartite graph** consists of two sets of vertices X and Y . The edges only join vertices in X to vertices in Y , not vertices within a set. (If there are r vertices in X and s vertices in Y then this graph is $K_{r,s}$.)

A **matching** is the pairing of some or all of the elements of one set, X , with elements of a second set, Y . If every member of X is paired with a member of Y the matching is said to be a **complete matching**.